

Pflichtenheft

CVPortal – Webbasiertes Lebenslauf- und Profilmanagementsystem

Projektbezeichnung: CVPortal **Version:** 1.0 **Erstellt am:** 08.05.2026

Systemarchitektur (Überblick)

Das Projekt besteht aus **zwei eigenständigen Spring Boot Applikationen**:

Komponente	Bezeichnung	Port	Aufgabe
cvportal-backend	REST-API Backend	8080	Datenhaltung, Geschäftslogik, Authentifizierung
cvportal-frontend	Web-Frontend	8081	Benutzeroberfläche, ruft Backend-API per HTTP auf

Die Trennung der beiden Projekte erfordert eine explizite **CORS-Konfiguration** im Backend, da der Browser Anfragen von **localhost:8081** an **localhost:8080** als Cross-Origin-Anfragen behandelt.

```

Browser (Port 8081)
  | HTTP-Request (Thymeleaf-Seite)
  ▼
cvportal-frontend (Spring Boot, Port 8081)
  | REST-Call via JavaScript fetch() / RestTemplate
  | → Origin: http://localhost:8081
  ▼
cvportal-backend (Spring Boot, Port 8080)
  | CORS-Header: Access-Control-Allow-Origin: http://localhost:8081
  ▼
H2 Database (embedded im Backend)
  
```

1. Zielbestimmung

1.1 Musskriterien

- **M01** – Das Backend stellt eine REST-API bereit, die alle Datenzugriffe kapselt. Das Frontend darf nicht direkt auf die Datenbank zugreifen.
- **M02** – Das Backend konfiguriert CORS explizit, sodass Anfragen vom Frontend (andere Origin) zugelassen werden.
- **M03** – Benutzer können sich am System registrieren und anmelden. Die Authentifizierung erfolgt über JWT (JSON Web Token).
- **M04** – Jeder Teilnehmer kann seinen Lebenslauf in strukturierter Form erfassen und bearbeiten (Stammdaten, Berufserfahrung, Ausbildung, Kenntnisse, Sprachen).
- **M05** – Administratoren (Berater) können alle Lebensläufe aller Teilnehmer einsehen.
- **M06** – Jeder Teilnehmer hat eine öffentlich zugängliche Web-Visitenkarte (**/card/{benutzername}**), die ohne Login abrufbar ist und wesentliche Profildaten sowie einen Link zum vollständigen Lebenslauf

enthält.

- **M07** – Der vollständige Lebenslauf ist über eine eigene öffentliche URL abrufbar (`/cv/{benutzername}`), sofern der Teilnehmer die Sichtbarkeit auf "öffentlich" gesetzt hat.
- **M08** – Das Frontend stellt alle Funktionen über ein responsives Web-Interface bereit (Bootstrap 5).
- **M09** – Alle Dateneingaben werden serverseitig validiert (Bean Validation).
- **M10** – Passwörter werden Argon2-gehasht gespeichert; JWT-Tokens sind zeitlich begrenzt (Ablauf nach 8 Stunden).

1.2 Wunschkriterien

- **W01** – QR-Code auf der Web-Visitenkarte, der auf die Visitenkarten-URL verweist (generiert via ZXing-Library).
- **W02** – Lebenslauf als PDF exportieren (serverseitig via iText oder Flying Saucer).
- **W03** – Teilnehmer können ihr Profil-Foto hochladen (als Base64 in der DB gespeichert).
- **W04** – Admin-Dashboard mit Übersicht: Anzahl Teilnehmer, Anzahl vollständiger Lebensläufe, zuletzt aktualisierte Profile.
- **W05** – Mehrsprachigkeit der Lebenslaufinhalte (Deutsch / Englisch parallel erfassbar).

1.3 Abgrenzungskriterien

- **A01** – Keine direkte Integration mit externen Jobbörsen oder AMS-Systemen.
- **A02** – Kein Echtzeit-Chat oder Kommentarfunktion zwischen Berater und Teilnehmer.
- **A03** – Kein automatischer CV-Import aus Word- oder PDF-Dateien.
- **A04** – Die Anwendung ist für den internen BBRZ-Betrieb konzipiert, nicht für den öffentlichen Interneteinsatz (kein HTTPS im Pflichtumfang).
- **A05** – Keine native Mobile App – nur responsives Web.

2. Produkteinsatz

2.1 Anwendungsbereiche

- Strukturierte Erstellung und Pflege von Lebensläufen im Rahmen von BBRZ-Ausbildungsmaßnahmen
- Digitale Portfolios für Teilnehmer bei der Stellensuche
- Übersicht für Berater über den Dokumentationsstand der Teilnehmer

2.2 Zielgruppen

Rolle	Beschreibung
Admin / Berater	Kann alle Teilnehmerprofile einsehen, verwalten und Benutzer anlegen
Teilnehmer	Erfasst und pflegt den eigenen Lebenslauf, steuert Sichtbarkeit
Externer Betrachter	Ruft öffentliche Visitenkarte / Lebenslauf ohne Login auf (nur Lese-Zugriff)

2.3 Betriebsbedingungen

- Betrieb im lokalen Netzwerk (BBRZ-intern)
- Beide Spring Boot Applikationen laufen auf demselben Server oder Entwicklerrechner
- Zugriff über Standard-Webbrowser; kein Plugin erforderlich

3. Produktumgebung

3.1 Software (Laufzeit)

Komponente	Version
Java Runtime Environment	21 (LTS) oder höher
Webbrowser	Chrome 110+, Firefox 110+, Edge 110+
Betriebssystem	Windows 10/11, Linux, macOS

3.2 Hardware (Mindestanforderungen)

Ressource	Mindest	Empfohlen
CPU	2 Cores, 1.5 GHz	4 Cores, 2 GHz
RAM	1 GB frei	2 GB frei (beide Apps laufen gleichzeitig)
Festplatte	300 MB	1 GB

3.3 Produktschnittstellen

Schnittstelle	Beschreibung
REST-API (intern)	Frontend → Backend über HTTP/JSON, Port 8080
H2-Datenbankschnittstelle	Backend → H2 via JDBC / Spring Data JPA
Browser-HTTP	Benutzer → Frontend über HTTP, Port 8081
QR-Code-Generierung	ZXing-Bibliothek intern im Backend (W01)
CORS-Header	Backend sendet Access-Control-Allow-Origin an Browser

4. Produktfunktionen

F01 – Registrierung und Login

Neue Teilnehmer können sich mit Benutzernamen, E-Mail und Passwort registrieren. Nach dem Login erhält das Frontend ein JWT, das bei jedem weiteren API-Aufruf im **Authorization: Bearer**-Header mitgesendet wird.

F02 – Lebenslauf erfassen und bearbeiten (Teilnehmer)

Der Teilnehmer füllt seinen Lebenslauf in mehreren Abschnitten aus:

- **Stammdaten:** Name, Adresse, Telefon, E-Mail, Geburtsdatum, Berufsbezeichnung
- **Berufserfahrung:** Einträge mit Firma, Position, Zeitraum, Beschreibung (mehrere möglich)
- **Ausbildung:** Schule/Institution, Abschluss, Zeitraum (mehrere möglich)
- **Kenntnisse:** Fähigkeiten mit Selbsteinschätzung (z.B. Java – Fortgeschritten)
- **Sprachen:** Sprache + Niveau (z.B. Englisch – B2)
- **Sichtbarkeit:** Öffentlich / Privat (steuert, ob **/cv/{benutzername}** zugänglich ist)

F03 – Lebenslauf anzeigen (öffentliche Ansicht)

Die URL `/cv/{benutzername}` liefert eine druckfreundliche Ansicht des Lebenslaufs, sofern dieser auf "öffentlich" gesetzt ist. Ohne Login abrufbar, kein JWT nötig (öffentlicher Endpunkt im Backend).

F04 – Web-Visitenkarte (öffentliche Ansicht)

Die URL `/card/{benutzername}` zeigt eine kompakte, visuell ansprechende Visitenkarte mit:

- Name und Berufsbezeichnung
- Kontaktdaten
- Kurzprofil (aus Stammdaten)
- Profilfoto (falls hinterlegt, W03)
- QR-Code mit Link zur Visitenkarte (W01)
- "Vollständigen Lebenslauf anzeigen"-Button (nur wenn Lebenslauf öffentlich)

F05 – Berater-Übersicht (Admin)

Der Admin sieht alle registrierten Teilnehmer in einer Tabelle mit Name, Datum der letzten Aktualisierung und Vollständigkeitsgrad des Lebenslaufs. Direkter Link zur Profilseite jedes Teilnehmers.

F06 – Benutzerverwaltung (Admin)

Der Admin kann Benutzer deaktivieren und Rollen ändern (Teilnehmer ↔ Admin).

F07 – CORS-Demonstration (technisches Kernmerkmal)

Das Backend konfiguriert CORS über `@CrossOrigin` auf Klassen-Ebene oder zentral via `WebMvcConfigurer`. Das Frontend sendet Requests von einer anderen Origin (Port 8081) und empfängt die korrekten CORS-Response-Header. Im Entwicklungsmodus wird dies durch die H2-Konsole und Browser-DevTools sichtbar gemacht.

F08 – QR-Code generieren (Wunschkriterium W01)

Das Backend stellt einen Endpunkt `/api/card/{benutzername}/qr` bereit, der einen QR-Code als PNG-Bild liefert. Der QR-Code enkodiert die URL der Visitenkarte. Generierung via ZXing Core-Bibliothek.

F09 – PDF-Export (Wunschkriterium W02)

Über einen Button im Frontend wird der Lebenslauf als formatiertes PDF heruntergeladen. Das Backend generiert das PDF serverseitig und liefert es als `application/pdf`-Response.

5. Produktdaten

Backend-Entitäten

User

- id (Long, PK)
- username (String, unique, not null)
- email (String, unique, not null)

- password (String, Argon2, not null)
- role (Enum: ADMIN, TEILNEHMER)
- active (Boolean)
- createdAt (LocalDateTime)

CurriculumVitae (1:1 zu User)

- id (Long, PK)
- user (User, FK, unique)
- jobTitle (String)
- phone (String)
- address (String)
- birthDate (LocalDate)
- summary (String, Kurzprofil)
- profilePhotoBase64 (String, nullable)
- publicVisible (Boolean, default: false)
- lastUpdated (LocalDateTime)

WorkExperience (n:1 zu CurriculumVitae)

- id (Long, PK)
- cv (CurriculumVitae, FK)
- company (String)
- position (String)
- startDate (LocalDate)
- endDate (LocalDate, nullable – "bis heute")
- description (String)
- sortOrder (Integer)

Education (n:1 zu CurriculumVitae)

- id (Long, PK)
- cv (CurriculumVitae, FK)
- institution (String)
- degree (String)
- fieldOfStudy (String)
- startDate (LocalDate)
- endDate (LocalDate, nullable)
- sortOrder (Integer)

Skill (n:1 zu CurriculumVitae)

- id (Long, PK)
- cv (CurriculumVitae, FK)
- name (String)
- level (Enum: ANFAENGER, GRUNDKENNTNISSE, FORTGESCHRITTEN, EXPERTE)

Language (n:1 zu CurriculumVitae)

- id (Long, PK)
- cv (CurriculumVitae, FK)

- language (String)
- level (Enum: A1, A2, B1, B2, C1, C2, MUTTERSPRACHE)

Datenmenge (Schätzung)

- User: 200 Einträge
- CV-Stammdaten: 200 Einträge
- WorkExperience: ≈ 3 pro CV $\rightarrow$ 600 Einträge
- Education: ≈ 2 pro CV $\rightarrow$ 400 Einträge
- Skills: ≈ 5 pro CV $\rightarrow$ 1.000 Einträge
- Languages: ≈ 2 pro CV $\rightarrow$ 400 Einträge

6. Produktleistungen

- **API-Antwortzeit:** < 500ms bei bis zu 50 gleichzeitigen Benutzern im LAN
- **Seitenaufbau Frontend:** < 2 Sekunden (inkl. API-Calls)
- **QR-Code-Generierung:** < 1 Sekunde
- **PDF-Export:** < 3 Sekunden pro Dokument
- **JWT-Gültigkeit:** 8 Stunden (danach erneuter Login erforderlich)
- **Datenpersistenz:** H2 im Datei-Modus, kein Datenverlust bei Neustart

7. Benutzeroberfläche

Frontend-Seiten

Seite	URL (Frontend)	Zugänglich für
Login / Registrierung	/login , /register	Alle (nicht eingeloggt)
Eigenes Dashboard	/dashboard	Eingeloggte Teilnehmer
Lebenslauf bearbeiten	/cv/edit	Eigener Teilnehmer
Lebenslauf anzeigen	/cv/{username}	Öffentlich (wenn freigegeben)
Web-Visitenkarte	/card/{username}	Öffentlich immer
Admin-Übersicht	/admin/participants	Admin
Benutzerverwaltung	/admin/users	Admin

Gestaltungsprinzipien

- Bootstrap 5, responsiv (Mobile-First)
- Zweispaltiges Layout auf Desktop (Navigation links, Inhalt rechts)
- Visitenkarte: visuell hervorgehobenes Design (Karten-Optik, Farbakzent)
- Formularvalidierung: clientseitig (HTML5 required) + serverseitige Fehlermeldungen via Thymeleaf

CORS-Fluss (sichtbar im Browser)

Im Browser-Netzwerk-Tab ist bei jedem API-Aufruf der **OPTIONS**-Preflight-Request sowie der **Access-Control-Allow-Origin**-Antwortheader sichtbar – ein zentrales Lernziel des Projektes.

8. Qualitäts-Zielbestimmungen

Merkmal	Ziel	Maßnahme
Sicherheit	Kein unbefugter Datenzugriff	JWT-Validierung bei jedem geschützten Endpunkt; CORS-Whitelist
Korrektheit	Vollständige Validierung aller Eingaben	Bean Validation + GlobalExceptionHandler mit strukturierten Fehlerantworten
Wartbarkeit	Klare Trennung Backend / Frontend	Keine Datenbankzugriffe im Frontend; REST-API als einzige Schnittstelle
Testbarkeit	Unit- und Integrationstests	JUnit 5, Mockito, Spring Boot Test / MockMvc
Lesbarkeit	Clean Code	Schichten-Architektur, Javadoc auf allen public Methoden
Benutzerfreundlichkeit	Klar verständliche UI	Formular-Feedback, Fortschrittsanzeige CV-Vollständigkeit
Verfügbarkeit	Schneller Start	Beide Apps starten in < 30 Sekunden

9. Globale Testszenarien und Testfälle

Testfall 1 – Registrierung und JWT-Login

- **Vorbedingung:** Kein Benutzer mit diesem Namen vorhanden
- **Aktion:** POST `/api/auth/register` mit Benutzerdaten; dann POST `/api/auth/login`
- **Erwartetes Ergebnis:** HTTP 201 bei Registrierung; HTTP 200 mit JWT-Token beim Login

Testfall 2 – Geschützter Endpunkt ohne Token

- **Vorbedingung:** –
- **Aktion:** GET `/api/cv/me` ohne Authorization-Header
- **Erwartetes Ergebnis:** HTTP 401 Unauthorized

Testfall 3 – CORS-Preflight-Request

- **Vorbedingung:** Backend läuft auf Port 8080
- **Aktion:** Browser sendet OPTIONS-Request von `localhost:8081` an `localhost:8080/api/cv`
- **Erwartetes Ergebnis:** HTTP 200; Response-Header enthält `Access-Control-Allow-Origin: http://localhost:8081`

Testfall 4 – Lebenslauf anlegen und abrufen

- **Vorbedingung:** Teilnehmer ist eingeloggt (JWT vorhanden)
- **Aktion:** PUT `/api/cv/me` mit vollständigen Stammdaten

- **Erwartetes Ergebnis:** HTTP 200; Daten korrekt in DB gespeichert; GET `/api/cv/me` liefert identische Daten zurück

Testfall 5 – Öffentliche Visitenkarte ohne Login

- **Vorbedingung:** Teilnehmer mit Benutzername "muster" existiert
- **Aktion:** GET `localhost:8081/card/muster` ohne Login im Browser
- **Erwartetes Ergebnis:** Visitenkarte wird angezeigt; kein Redirect auf Login-Seite

Testfall 6 – Öffentlicher Lebenslauf (freigegeben)

- **Vorbedingung:** Teilnehmer hat `publicVisible = true` gesetzt
- **Aktion:** GET `localhost:8081/cv/muster` ohne Login
- **Erwartetes Ergebnis:** Vollständiger Lebenslauf wird angezeigt

Testfall 7 – Öffentlicher Lebenslauf (nicht freigegeben)

- **Vorbedingung:** Teilnehmer hat `publicVisible = false`
- **Aktion:** GET `localhost:8081/cv/muster` ohne Login
- **Erwartetes Ergebnis:** HTTP 403 / Meldung "Profil nicht öffentlich"

Testfall 8 – Admin sieht alle Profile

- **Vorbedingung:** Admin ist eingeloggt
- **Aktion:** GET `/api/admin/participants`
- **Erwartetes Ergebnis:** Liste aller Teilnehmer mit CV-Metadaten; HTTP 200

Testfall 9 – Teilnehmer kann nicht auf Admin-Endpunkt zugreifen

- **Vorbedingung:** Teilnehmer ist eingeloggt (Rolle: TEILNEHMER)
- **Aktion:** GET `/api/admin/participants` mit Teilnehmer-JWT
- **Erwartetes Ergebnis:** HTTP 403 Forbidden

Testfall 10 – QR-Code generieren (W01)

- **Vorbedingung:** Benutzer "muster" existiert
- **Aktion:** GET `/api/card/muster/qr`
- **Erwartetes Ergebnis:** HTTP 200; Content-Type: `image/png`; PNG-Bild korrekt generiert

Testfall 11 – Unit Test: JWT-Generierung und Validierung (automatisiert)

- **Typ:** JUnit 5 Unit Test
- **Beschreibung:** `JwtService.generateToken()` erzeugt valides Token; `JwtService.isValidToken()` gibt `true` zurück; abgelaufenes Token gibt `false`

Testfall 12 – Unit Test: CV-Vollständigkeitsgrad (automatisiert)

- **Typ:** JUnit 5 Unit Test im Service-Layer
- **Beschreibung:** `CvService.calculateCompleteness()` gibt 0% für leeres CV, 100% wenn alle Pflichtfelder und mind. je 1 Eintrag in WorkExperience, Education, Skill, Language vorhanden

10. Entwicklungsumgebung

10.1 Software

Tool	Version / Zweck
JDK	21 (LTS)
Spring Boot	4.x.x (beide Projekte)
Maven	3.6.x
IntelliJ IDEA	2025.x
Git	2.x
H2 Database	via Spring Boot Starter (nur im Backend)
Spring Security	7.x (JWT via <code>jjwt</code> -Bibliothek)
ZXing	3.5.x (QR-Code, W01)
Thymeleaf	via Spring Boot Starter (nur im Frontend)
Bootstrap	5.3.x
Browser DevTools	Chrome/Firefox – für CORS-Visualisierung

10.2 Hardware

- Entwickler-PC, mind. 8 GB RAM (beide Apps laufen gleichzeitig)
- Internetzugang für Maven-Dependencies (einmalig)

10.3 Entwicklungsschnittstellen

Schnittstelle	Zweck
<code>mvn spring-boot:run</code>	Start beider Applikationen (je in eigenem Terminal)
H2-Konsole (<code>/h2-console</code>)	Datenbankinspektion während Entwicklung (nur Backend, nur Dev-Profil)
REST-Client (z.B. Bruno, Postman)	Direktes Testen der Backend-API unabhängig vom Frontend
Git	Zwei Repositories (oder ein Mono-Repo mit zwei Maven-Modulen)

11. Ergänzungen

CORS – Technische Hintergründe (Lernziel)

Was ist CORS? CORS (Cross-Origin Resource Sharing) ist ein Sicherheitsmechanismus des Browsers. Sendet eine Webseite unter `http://localhost:8081` eine JavaScript-Anfrage an `http://localhost:8080`, blockiert der Browser diese standardmäßig. Das Backend muss explizit durch HTTP-Header mitteilen, welche Origins erlaubt sind.

Umsetzung im Projekt:

```
// cvportal-backend: CorsConfig.java
@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:8081")
            .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
            .allowedHeaders("*")
            .allowCredentials(false)
            .maxAge(3600);
    }
}
```

Öffentliche Endpunkte (/api/card/**, /api/cv/public/**)

JWT-Authentifizierungsfluss

1. POST /api/auth/login → Backend prüft Credentials → liefert JWT
2. Frontend speichert JWT im SessionStorage
3. Jeder folgende API-Call: Header "Authorization: Bearer <token>"
4. Backend-Filter validiert Token bei jedem Request
5. Nach 8h: Token abgelaufen → Frontend leitet auf Login-Seite um

Projektstruktur

```
cvportal/
├── cvportal-backend/           ← Maven-Projekt 1 (Port 8080)
│   ├── src/main/java/at/bbrz/cvportal/backend/
│   │   ├── config/            (SecurityConfig, CorsConfig, JwtConfig)
│   │   ├── controller/        (AuthController, CvController, CardController,
AdminController)
│   │   ├── dto/               (LoginRequest, JwtResponse, CvDto, CardDto, ...)
│   │   ├── entity/            (User, CurriculumVitae, WorkExperience, Education,
Skill, Language)
│   │   ├── exception/         (GlobalExceptionHandler, ResourceNotFoundException)
│   │   ├── repository/        (UserRepository, CvRepository, ...)
│   │   └── service/           (AuthService, JwtService, CvService, CardService,
QrService)
│   └── src/main/resources/
│       ├── application.properties
│       └── data.sql
└── cvportal-frontend/         ← Maven-Projekt 2 (Port 8081)
    ├── src/main/java/at/bbrz/cvportal/frontend/
    │   └── config/             (WebClientConfig - konfiguriert RestTemplate mit
```

```

Backend-URL)
  |   |— controller/      (PageController, CvPageController,
CardPageController)
  |   |— service/        (ApiClientService – alle Calls an das Backend)
  |   |— src/main/resources/
  |       |— application.properties (backend.url=http://localhost:8080)
  |       |— templates/          (Thymeleaf HTML-Templates)

```

Sicherheitskonzept

Maßnahme	Umsetzung
Authentifizierung	JWT (HS256, 8h Gültigkeit)
Passwörter	Argon2id
CORS	Whitelist: nur Frontend-Origin erlaubt
Autorisierung	Spring Security Method-Security (@PreAuthorize)
Eingabevalidierung	Bean Validation auf allen DTOs
SQL-Injection	Ausschließlich JPA/JPQL-Abfragen
XSS	Thymeleaf escaped standardmäßig; kein <code>th:utext</code> auf Benutzerdaten
Öffentliche Endpunkte	Klar definiert, nur Lesezugriff, keine sensiblen Daten

Konzept Weiterentwicklung: HTTPS (TLS), Rate-Limiting auf Login-Endpunkt, Refresh-Token-Mechanismus, Audit-Log.